YuvaSaarthi v2 - Technical Presentation & Architecture Guide

1. Project Overview & Problem Statement

YuvaSaarthi v2 is a highly contextual, multimodal national education assistant designed for Indian students. The fundamental problem it addresses is the lack of equitable, scalable, and personalized academic guidance for students, especially in lower-tier cities and rural areas. It bridges the gap between complex official documents (like syllabi, scheme notifications, exams) and the student by using advanced Natural Language Processing (NLP).

2. Core Technological Architecture

YuvaSaarthi moves away from basic API wrappers and employs a complete, robust Generative AI pipeline.

A. The Generation Layer (LLM)

We leverage **Groq's Llama-3.3-70b-versatile** models. Groq utilizes Linguistic Processing Units (LPUs) rather than GPUs, providing unprecedented inference speeds (often >800 tokens per second). This ensures real-time conversational capabilities even for complex educational queries.

B. The Retrieval-Augmented Generation (RAG) Layer

To prevent LLM hallucination (the AI making things up), YuvaSaarthi uses a local vector database:

- **Embeddings:** `sentence-transformers/paraphrase-multilingual-mpnet-base-v2` encodes raw text into high-dimensional numerical vectors.
- **ChromaDB:** A dedicated vector store that performs cosine similarity searches. When a user asks a question, the engine retrieves the top 3 most relevant textbook paragraphs or syllabus documents before querying the LLM.

C. The Multimodal & Translation Layer

- **Deep-Translator & LanguageDetector:** Automatically identifies input language and seamlessly translates to English for backend RAG processing, then back to the native language (Hindi, Tamil, Marathi, etc.) for the user.
- **Vision (Llama-3.2 Vision):** Processes images (diagrams/questions) converted to base64 via API.
- **Whisper & gTTS:** Handles local and cloud-based automatic speech recognition (ASR) and text-to-speech (TTS), breaking input barriers for marginalized users.

---

3. The 13 Advanced Features Implementation

1. **Socratic Dialogue Mode:** Injects dynamic behavioral prompts into the LLM context. When activated, the AI forces the model to hold back direct answers and instead guide the student using leading questions.
2. **Distress Detection & Helpline Routing:** The engine parses incoming strings against a dictionary of self-harm or severe stress keywords. If matched, it gracefully intercepts the LLM pipeline, executing an immediate early-return protocol to supply official helpline numbers (AASRA, iCall).
3. **Syllabus Gap Tracker:** Employs an LLM-based classification sub-routine that reads the user's query, maps it to predefined JSON syllabus chapters, and persists progress metrics in a local `SQLite` database.

4. **OCR "Point and Ask":** Converts uploaded local images (like textbook diagrams) to base64 binary strings and pushes them through a Vision Language Model (`llama-3.2-11b-vision`) payload to extract and explain content.
5. **Teach-Back Assessment:** An active pedagogical pipeline injection that prompts the student to restate learned concepts. The LLM acts as an evaluator, mapping the user's response against reference materials.
6. **Form-Filling Assistant:** Replaces complex HTML layout rendering by utilizing Python's `fpdf` library. It builds dynamically accessible PDF applications populated iteratively through a backend dictionary.
7. **Teacher-Assist Mode:** Introduces role-based intent routing. If the query detects "lesson plan," it bypasses normal RAG paths to execute a structured 5-part lesson generator optimized for instructors.
8. **Spaced Repetition Engine:** Implements the mathematically rigorous **SuperMemo-2 (SM-2)** algorithm within SQLite. It recalculates interval decay rates (Ease Factor) dynamically every time a student revisits a topic computationally.
9. **Native Language Mnemonic Generator:** Adjusts temperature and system prompts dynamically, injecting strict instructions to bind memory aids to regional cultural markers (like Bollywood, Cricket, Festivals).
10. **Exam Pattern Intelligence:** Overcomes LLM data-caps by injecting hardcoded JSON historical Previous Year Question (PYQ) weightage constraints directly into the context window, forcing deterministic, highly accurate strategy advice.
11. **MyScheme API Integration:** Mitigates unreliable third-party APIs by maintaining an active local SQLite cache of national scholarship criteria, performing LIKE-query fast searches for relevant financial aid.
12. **NEP 2020 Curriculum Mapping:** Evaluates classified query topics against a static multidimensional JSON dictionary of India's National Education Policy frameworks and appends exact stage/competency metadata.
13. **Voice-in / Voice-out (Speech Framework):** Combines Groq's local `whisper-large-v3` inference endpoint for Speech-to-Text with Python's `gTTS` wrapper, enabling pure voice interface interactions.

---

4. Examiner Q&A Guide (Defense Prep)

Q1: Why did you use local ChromaDB instead of managed cloud vectors like Pinecone? *Answer:* To eliminate recurring overhead costs, minimize completely API latency, and maintain strict data privacy control over our massive 3.2+ GBs of document embeddings. It ensures the app runs sustainably without external database dependencies.

Q2: How do you prevent the AI from giving wrong answers (hallucination)? *Answer:* Through strict Retrieval-Augmented Generation (RAG). The LLM's system prompt strictly instructs it to answer ONLY based on the provided context retrieved from ChromaDB.

Q3: Doesn't translating queries degrade the educational accuracy? *Answer:* We map native questions to English purely for the internal RAG retrieval (since our textbooks are embedded primarily in English). We then use the LLM to generate the answer natively or translate the English context back. This vastly improves vector similarity search precision while retaining user comfort.

Q4: How does your Spaced Repetition algo actually calculate when to show a topic? *Answer:* We use the SM-2 algorithm. It assigns an initial 'Ease Factor' of 2.5. Based on user recall quality, we adjust the interval (Days = Interval * Ease Factor). If they struggle, the interval resets to 1; if they succeed, it expands exponentially.

Q5: What happens if Groq API goes down? *Answer:* The architecture is designed modularly. Because we use the LangChain abstraction wrapper, we can hot-swap `ChatGroq` for local models (like `Ollama`) or `OpenAI` by modifying just 2 lines in `backend/llm_handler.py`.

Q6: Why is the Vector DB explicitly excluded from your Git commits? *Answer:* ChromaDB relies on machine-specific SQLite binaries which break operating system compatibilities and exceed standard 1GB GitHub limitations. Good DevOps practice mandates pushing code and raw resources separately, dynamically re-building databases via our ingest scripts.